Project Report Template

Project Title

Galactic Defender – Assembly Language Game

Group Members

Neeraj Khemani – 24K-0548

Muhammad Ali Naqvi – 24K-1004

Submission Date

26 November 2025

---

1. Executive Summary

- **Overview**:

  "Galactic Defender" is a text-based shooter game developed in **Assembly Language (MASM + Irvine32)**. The project demonstrates real-time input handling, enemy movement, shooting mechanics, and scoring. All major objectives like player controls, enemy AI, collisions, and level progression were successfully achieved.

- **Key Findings**:

  - Successfully implemented real-time player movement and shooting using keyboard event polling.
  - Designed enemy ship with movement patterns and bullet firing mechanics.
  - Implemented collision detection between bullets, enemies, and the player.
  - Developed a working scoring system and level progression mechanism.
  - Demonstrated Assembly-level concepts such as stack usage, registers, loops, arrays/structures, and efficient use of procedures.

2. Introduction

- **Background**:

  Most beginner Assembly projects are static. This project introduces real-time interactivity and demonstrates how low-level programming can be used to build a complete game.
  This project is relevant to COAL because it uses:

  - Low-level CPU instructions
  - Direct memory access
  - Input handling at the Assembly level
  - Procedures, stacks, and registers

- **Project Objectives**:

  - Develop a real-time text-based shooter game using MASM and Irvine32.
  - Implement player movement, shooting, enemy spawning, and enemy firing.
  - Create collision detection for bullets vs enemies and enemies vs player.
  - Add three difficulty levels with progressive speed and challenge.

3. Project Description

- **Scope**:

  Included Features:
  - Three levels: Beginner, Intermediate, Advanced
  - Player spaceship movement (left/right)
  - Player bullet firing
  - Enemy movement and firing
  - Collision detection
  - Scoring system and level transitions
  - Console-based game interface with text graphics

  Excluded Features:
  - Advanced graphical rendering
  - Multiplayer support
  - High-score saving system

- **Technical Overview**:
  - **Assembler:** MASM (Microsoft Macro Assembler)
  - **Library:** Irvine32
  - **IDE:** Visual Studio
  - **Platform:** Windows (x86)

4. Methodology

- **Approach**:

  We used incremental development, starting with basic movement, then bullets, enemies, and finally collisions and levels.

  Roles and Responsibilities:

  1. **Neeraj Khemani (24K-0548):**
     - Player movement and shooting
     - Bullet handling
     - Collision detection
     - Scoring system
     - Level progression
  2. **Muhammad Ali Naqvi (24K-1004):**
     - Enemy spawning and AI movement
     - Enemy bullet firing
     - Game UI and screen formatting
     - Game over, pause, and win screens

5. Project Implementation

- **Design and Structure**:
  - Player movement & shooting
  - Enemy movement & firing
  - Bullet management
  - Level system
  - Score display
  - Pause (P) and Exit (ESC) input handling

- **Functionalities Developed**:
  - Real-time keyboard input
  - Player/enemy rendering using text
  - Collision detection
  - Level transitions
  - Game over & win screens

- **Challenges Faced**:
  - Reducing screen flicker
  - Managing multiple bullets
  - Timing enemy movement
  - Maintaining cursor positions

6. Results

- **Project Outcomes**:
  - A complete working Assembly game
  - Three difficulty levels
  - Fully working shooting, enemy AI, and collision system
  - Smooth player controls and real-time gameplay

- **Testing and Validation**:

  Movement, bullets, collisions, and level transitions were tested thoroughly with boundary and stress tests.

7. Conclusion

- **Summary of Findings**:

  The game successfully demonstrates real-time Assembly programming, applying COAL concepts such as input handling, cursor control, loops, and memory management. All goals were achieved.

- **Final Remarks**:

  The project provided hands-on experience with low-level logic and improved understanding of how CPU instructions translate into real-time interactive behavior